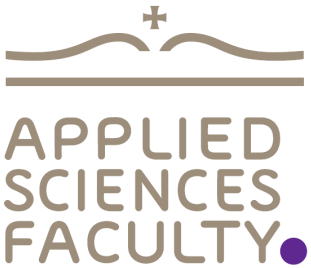
C++ — лекція 2. Інструкції (statement) та оператори





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Інструкції та оператори

- Програма на C/C++ складається із операторів, що віддають команди та **оголошень, (declarations)**, що описують нові сутності — змінні, функції, тощо.
- Блок є способом об'єднати оператори та оголошення в єдиний, складений оператор.
- Оператор — найменша самостійна частина програми.
- Те, на що діють оператори, називається операндами.
- Комбінація операторів — вираз, expression[1], що завершується крапкою із комою, є самостійною інструкцією (statement).
- Всі вирази повертають **значення**, яке можна **надати**, скажімо, якійсь змінній.

Приклад виразу:

```
c = a + b;
d1 = c1 = a1 + b1;
```

[1] Формальні означення див. https://en.cppreference.com/w/cpp/language/expressions.

Операторори, в порядку спадання пріоритету

1. Керування областю видимості.
   - Вказання області видимості, включаючи глобальну:

`class-name::member`, `namespace-name::member`, `::globalname`

2.
   - Вибір члена (поля або метода) класу або структури:

`object.member`, `pointer->member`

3.
   - Доступ за індексом: `pointer[expression]`
   - Виклик функції: `function_name(expression-list)`
   - Конструювання: `type_name(expressions-list)`, `type_name{expressions-list}` (C++11).
   - Постфіксний інкремент `lvalue++` та декремент `lvalue--`.
   - Визначення типу: **typeid** `(type)` та **typeid** `(expression)`
   - Приведення типів: **static_cast**`<type>(expression)`, **dynamic_cast**`<type>(expression)`, **reinterpret_cast**`<type>(expression)`, **const_cast**`<type>(expression)`,

4.

- Розмір об'єкта чи типу: **sizeof**`(expression)`, **sizeof**`(type)`
- Префіксний інкремент та декремент: `++lvalue` та декремент, `--lvalue`
- Логічне заперечення[2] (NOT): `!expression`
- Побітове заперечення (bitwise NOT): `~expression`
- Унарний плюс та мінус: `+expression`, `-expression`
- Адреса: `&expression`
- Розіменування: `*expression`
- Виділення та звільнення пам'яті: `new`, `delete`, `new[]`, `delete[]`
- Приведення типів в стилі C: `(type)expression`
- `co_await` (C++20)

5.

- Вибір члена (поля або метода) за вказівником на клас або структуру: `object.*pointer_to_member`, `pointer->*pointer_to_member`

[2] Logical negation, logical NOT: `https://en.cppreference.com/w/cpp/language/operator_logical`.

6

- Ділення, множення, остача: `expression * expression`, `expression / expression`, `expression % expression`.

7

- Додавання, віднімання: `expression + expression`, `expression - expression`.

8

- Побітовий зсув (та ввід-вивід): `expression << expression`, `expression >> expression`.

9

- Three-way comparison operator, він же – spaceship operator (C++20): `<=>`.

10

- Оператори відношень, більше, більше або дорівнює, менше, менше або дорівнює : `expression > expression`, `expression >= expression`, `expression < expression`, `expression <= expression`.

11

- Операторори дорівнює та не дорівнює: `expression == expression`, `expression != expression`.

12

- Побітове І (AND): `expression & expression`.

13

- Побітове Виключне АБО (XOR): `expression ^ expression`.

14

- Побітове АБО (OR): `expression | expression`.

15

- Логічне І (AND): `expression && expression`.

16

- Логічне АБО (OR): `expression || expression`.

17

- Умовний тернарний оператор: `expression ? expression : expression`.

18

- Присвоєння та Оператори разом із присвоєнням: `lvalue = expression`, `lvalue *= expression`, `lvalue /= expression`, `lvalue %= expression`, `lvalue += expression`, `lvalue -= expression`, `lvalue < <= expression`, `lvalue > >= expression`, `lvalue &= expression`, `lvalue |= expression`, `lvalue ^= expression`.

19

- Генерація виключень: **throw** `expression`.
- `co_yield` (C++20)

20

- Оператор послідовності, кома: `expression, expression`.

- Дужки, самі по собі операторами не є[3], вони лише дозволяють керувати пріоритетом.
- ◇ Сумніваєтеся[4] – поставте дужки.
- Сплутати пріоритет ряду операторів важко, або неможливо: `const_cast`, `static_cast`, `dynamic_cast`, `reinterpret_cast`, `typeid`, `sizeof...`[5], `noexcept`[5], `alignof`[5].
- Деякі оператори мають альтернативні форми: `and` замість `&&`; `or` замість `||`; `not` замість `!`.
- ◇ Базово, вони призначені для використання на терміналах без відповідних спец-символів, тому не рекомендовані до використання.
- ◇ MSVC їх не підтримує[6].
- Аргументи sizeof, typeid, noexcept та decltype хоча й компілюються, але ніколи не обчислюються. Це так звані unevaluated expressions. Вони не генерують коду, побічні ефекти відповідних виразів відсутні. Виняток: поліморфні аргументи typeid.

[3] Не плутати із operator(), перевантаженням оператора виклику функції.

[4] В свої чи колег пам'яті або уважності.

[5] C++11.

[6] Див. `https://stackoverflow.com/a/24414345` – є ключі, що вмикають підтримку.

Скорочена таблиця пріоритетів:

- Постфіксний інкремент та декремент: var++, `var--`.
- Префіксний інкремент та декремент: ++var, `--var`; унарний плюс та мінус — для отримання від'ємного та додатного значень (-var, +5). Побітове заперечення, '~'. Логічне заперечення, '!'. Оператор розіменування вказівника, '*'. Оператор взяття адресу: '&'. Оператор розміру, `sizeof`.
- Ділення, множення, остача: '*', '/', '%'.
- Додавання, віднімання: '+', '-'.
- Оператор бітових зсувів або виводу-вводу: '«', '»'.
- Оператори порівняння: '<', '<=', '>', '>='.
- Оператори порівняння: '==', '!='.
- Побітове AND: '&'.
- Побітове XOR: '^'.
- Побітове OR: '|'.
- Логічне AND: '&&'.
- Логічне OR: '||'.
- Тернарний оператор: '? :'
- Оператори присвоєння: '=', '+=', '-=', '*=', '/=', '%=', '>>=','<<=','&=','^=','|='.

Асоціативність

- Всі оператори присвоювання правоасоціативні (right-to-left associativity), тобто a=b=c означає a=(b=c).
- Решта бінарних операторів — лівоасоціативні (left-to-right associativity): a+b+c це те ж, що і (a+b)+c.
- Асоціативність унарних операторів:
  - ◇ Префіксні унарні оператори правоасоціативні (right-to-left): `delete ++*p` це `delete(++(*p))`.
  - ◇ Постфіксні унарні оператори лівоасоціативні (left-to-right): `a[1][2]++` це `((a[1])[2])++`.
- Зауважте, `*pobj.data` це `*(pobj.data)`, тому, якщо це не те, що вам потрібно (та й тоді краще дужки поставити), замість потворного синтаксису `(*pobj).data` варто писати `pobj->data`
- В чистому C, на відміну від C++, тернарний оператор має вищий пріоритет, ніж присвоєння.
- Див. також `https://en.cppreference.com/w/cpp/language/operator_precedence`, але там згруповано деякі блоки, розділені вище, щоб потім виправдовуватися[7].

[7] Цитуючи: *Note that the associativity is meaningful for member access operators, even though they are grouped with unary postfix operators: a.b++ is parsed (a.b)++ and not a.(b++).*

Порядок обчислення

- Порядок обчислення підвиразів не визначений, за кількома винятками.
- Наприклад, не відомо, що буде обчислено першим:
  - Виклик f() чи g() в : `c = f(a)+g(b)`.
  - Вираз `[i]` чи i++ в: `arr[i] = i++`.
  - Зокрема, в різних місцях коду цей порядок може бути різним.
- Ремарка: порядок обчислень (під час виконання коду) не має відношення до асоціативності (яка впливає на компіляцію).
  - Для виразу f() + g() + h() першим буде виконуватися додавання результатів f() і g(), але сам порядок виклику функцій не фіксований – першим може взагалі бути обчисленим h(), потім g() і тоді f().
- Зроблено це, щоб дати компілятору додатковий простір для оптимізації.
- Єдиний виняток: оператори коми (','), логічного AND ('&&') та логічного OR ('||') обчислюють свої операнди зліва направо, а тернарний оператор – спершу умову, потім 2-й або 3-й аргумент.
  - Більше того, для вбудованих && та || обчислення відбувається в скороченій формі — другий аргумент обчислюється лише тоді, коли результат не є однозначним з першого.
  - Ці правила перестають виконуватися, якщо перевантажити відповідні оператори (але див. C++17).

Порядок обчислення – Sequenced-before rules C++11

- Трішки формальніше про термінологію, яка використовувалася вище.
- Виконання кожного виразу складається із двох частин – обчислення значення, що повертається та виконання побічних дій.
  - Побічні дії це: будь-яке звертання до volatile-об'єктів, зміна об'єктів у пам'яті, виконання вводу-виводу.
- Якщо дія A є впорядкованим перед B (A is sequenced before B) – обчислення A буде завершено до того, як почнеться обчислення B.
- Якщо ні A не є перед B ні B перед A, то існує два варіанти:
  - Виконання A і B є взаємно невпорядкованим (unsequenced) – вони можуть обчислюватися в довільному порядку чи навіть перекриватися.
  - A і B є невизначено впорядкованими (indeterminately sequenced) – спершу може обчислюватися або A або B, але завжди один буде повністю обчислено до завершення іншого.
- До C++11 використовувалося поняття sequence point. Детальніше див. `https://en.cppreference.com/w/cpp/language/eval_order`.

Порядок обчислення – Sequenced-before rules C++11/C++14

- Будь-який повний вираз – який завершується крапкою з комою, або блок умови з for/while/if, або ініціалізатор чи константний вираз, є повністю впорядкований перед наступним виразом, включаючи всі присвоєння та виклики деструкторів тимчасових об'єктів.
- Обчислення операндів оператора впорядковане перед обчисленням результату. Побічних ефектів це не стосується.
- Всі вирази-операнди функцій (включаючи побічні ефекти) впорядковані перед першим виразом тіла функції.
- C++14 наголосив, що копіювання об'єкта, який повертається з функції, відбувається перед знищенням тимчасових об'єктів виразу return, а воно – перед знищенням локальних об'єктів функції.

Порядок обчислення – Sequenced-before rules C++17

- C++17 трохи обмежив свободу старіших компіляторів по впорядкуванню обчислення виразів.
- До C++17, обчислення аргументів виразу new було невизначено впорядкованими відносно виклику оператора new – виділення пам'яті (викликом оператора new). В C++17 виділення пам'яті – строго перед обчисленням операндів, це зняло ряд проблем із розумними вказівниками.
- Вираз обчислення функції, яка викликається, відбувається до обчислення її аргументів, включаючи задані по замовчуванню.
- Обчислення аргументів функції є невизначено впорядкованими – порядок невідомий, але перекриватися не будуть.
- Навіть перевантажені оператори зберігають гарантії вбудованих операторів (важливо це для &&, ||, ,), якщо виклик здійснено з використанням операторів (а не як функцію – operator@@(...)).
- В операторі індексування E1[E2], E1 впорядковане перед E2.
- Це ж стосується E1.*E2 та E1->*E2.
- Це ж стосується E1<<E2 та E1>>E2.
- В E1 = E2 та E1 @= E2, де @ – будь-яка із можливих операцій, E2 впорядкований перед E1.
- В списку ініціалізації порядок обчислень – indeterminately-sequenced, як і для аргументів функції.
- Зауважте – раніше це все не гарантувалося.

Порядок обчислення – Undefined behavior

- Якщо дві операції над тим же скалярним об'єктом (грубо кажучи – не масивом і не структурою/класом) не впорядковані (unsequenced) – це невизначена поведінка.
- Аналогічно, якщо побічний ефект виразу змінює об'єкт і не є впорядкованим з звертанням до об'єкту, який він змінює (++i + i).
- При тому, для операторів впорядкування побічних ефектів може бути іншим, ніж для обчислення результатів.
- Простий приклад: двічі змінювати один об'єкт в одному виразі – невизначена поведінка в більшості випадків.
- Приклади:

```
i = ++i + 2;       // undefined behavior until C++11
i = i++ + 2;       // undefined behavior until C++17
f(i = -2, i = -2); // undefined behavior until C++17
f(++i, ++i);       // undefined behavior until C++17, unspecified after
i = ++i + i++;     // undefined behavior

cout << i << i++; // undefined behavior until C++17
a[i] = i++;       // undefined behavior until C++17
n = ++i + i;      // undefined behavior
```

Приклад помилкового коду:

```
void fn(int x, int y){
    cout << x << ", " << y << endl;
}

    .....
    int a=5;
    fn(++a,++a);
    .....
    int ar[5] = {1,1,1,1,1};
    int k = 2;
    ar[k]=++k;
    for(int i=0;i<5;++i)
    {
        cout << "ar[" << i << "]=" << ar[i] << endl;
    }
```

Результат його роботи:

Open Watcom, різні комбінації опцій

```
a=7
Running fn(++a,++a):
9, 8
```

Icc, з/без оптимізації

```
a=7
Running fn(++a,++a):
9, 8
```

tcc

```
a=7
Running fn(++a,++a):
8, 9
```

gcc, з/без оптимізації, різні OS та версії

```
a=7
Running fn(++a,++a):
9, 9
```

VS

```
a=7
Running fn(++a,++a):
9, 9
```

icc, з/без оптимізації

```
a=7
Running fn(++a,++a):
8, 9
```

Практично будь-який компілятор постарається вберегти вас від таких проблем, давши попередження, але покладатися на це не можна — в складніших випадках він і сам не зможе розібратися.

Попередження компілятора для програми із попереднього слайду:

```
GCC post-C++17 (зверніть увагу на повтор):
<path>\l02_evaluation_order\l02_evaluation_order_main.cpp:24:12: warning:
 operation on 'a' may be undefined [-Wsequence-point]
   24 |     fn(++a,++a);
      |            ^~~
<path>\l02_evaluation_order\l02_evaluation_order_main.cpp:24:12: warning:
 operation on 'a' may be undefined [-Wsequence-point]

Clang post-C++17:
<path>/l02_evaluation_order/l02_evaluation_order_main.cpp:24:8: warning:
 multiple unsequenced modifications to 'a' [-Wunsequenced]
    fn(++a,++a);
       ^   ~~

MSVC pre-C++17:
<path>l02_evaluation_order_main.cpp:21:16: warning:
 operation on 'a' may be undefined [-Wsequence-point]
     fn(++a,++a);
                ^
<path>\l02_evaluation_order_main.cpp:26:14: warning:
 operation on 'k' may be undefined [-Wsequence-point]
```

Бітові зсуви – невизначена поведінка

- Для бітових зсувів, як ліворуч, так і праворуч, якщо величина зсуву від'ємна, чи рівна або більша за кількість біт у аргументі – поведінка не визначена.
- Для знакових аргументів зсуву ліворуч, якщо значення додатне, але результат зсуву не поміститься – поведінка невизначена.
- Для зсуву праворуч від'ємних значень, поведінка – implementation-defined.
- Причина – процесори дуже по різному ведуть себе у випадках таких зсувів, тому, замість шукати спільний знаменник – який сповільнюватиме згенерований код для багато яких ISA, в стандарті відмовляються опрацьовувати такі випадки.
- ◇ `https://devblogs.microsoft.com/oldnewthing/20230904-00/?p=108704` – "Just for fun: What happens when you shift a register by more than the register size?"

&&	!alive	alive
!dead		
dead		

Spaceship operator – three-way comparison operator

- Повний опис, як порівнювати два об'єкти, включає (до C++20) оголошення шести операторів.
- При тому, всі шість потрібно для кожного перевантаження аргументів – скажімо, для класу раціональних чисел ratio_t потрібно 18 комбінацій: 6 для op(ratio_t, ratio_t), 6 для: op(ratio_t, int) і 6 для: op(int, ratio_t).
- Це дуже марудно і нецікаво писати.
- Деякі конструкції, типу std::rel_ops (`https://en.cppreference.com/w/cpp/utility/rel_ops/operator_cmp`) чи Boost.utility.operators (`https://www.boost.org/doc/libs/1_79_0/libs/utility/doc/html/utility/utilities/operators.html`) пропонують вирішення, але все ще є ерзацом.
- Для систематичного вирішення в C++20 додали оператор тришляхового порівняння[8]: `operator<=>`.
- Його можна використовувати для безпосереднього порівняння, але цікавий він тим, що вміє генерувати більшість комбінації звичних операторів порівняння автоматично.

[8] Схожі оператори є в інших мовах: Perl, PHP, Ruby тощо, а поведінка їх схожа на функції типу strcmp() в C.

Найпростіший приклад

- Якщо зробити так:

```
struct C1{
    int a, b;
    auto operator<=>(const C1&) const = default;
};
```

то автоматично і коректно[9] працюватимуть:

```
C1 a1{1, 2}, a2 {2, 1};
static_cast<void>(a1<a2, a1<=a2, a1==a2, a1!=a2, a1>=a2, a1>a2);
```

- Також, можна безпосередньо скористатися spaceship operator:

```
(a1 <=> a2) < 0  // true if a1 <  a2
(a1 <=> a2) > 0  // true if a1 >  a2
(a1 <=> a2) == 0 // true if a1 == a2
```

[9] Залежно від ситуації, звичайно.

- Порівняння по замовчуванню для класу відбувається спочатку для базових класів, починаючи зліва, спершу – вглиб.
- Потім порівнюються члени класу у лексикографічному порядку.
- Масиви порівнюються зліва направо, якщо вони багатомірні – рекурсивно заглиблюючись у кожен вимір.
- Якщо масиви різного розміру, порівняння на рівність (==) відбувається за розміром.
- Інші порівняння – поелементно. Якщо всі наявні елементи співпадають, більшим вважається довший масив.
  - ◇ Це той же порядок, в якому об'єкти створюються.
  - ◇ У прикладі вище – спершу порівнюються члени a, потім b.
- Оператор тришляхового порівняння не означається автоматично – щоб його варіант по замовчуванню створився, потрібно про це сказати за допомогою =default.
- Зауваження: таке написання сильно спрощує створення типізованих обгорток над числовими типами, наприклад, для контролю розмірності.

- Якщо оголошено <=>, оператор == створюється неявно, але його можна створити і вручну – якщо порівняння робиться по іншому:

```
struct C2{
    int a, b;
    auto operator<=>(const C2&) const = default;
    bool operator==(const C2& rhs) const{
        return (a == rhs.a) && (b == rhs.b);
    }
}
```

- Оператор <=> може бути як членом, так і незалежною функцією-другом.
- Якщо <=> оголошено, операції порівняння `a @ b`, де `@` – один із операторів порівняння, перетворюються на `(a <=> b) @ 0`.
- Якщо такий код є некоректним, але є неявний конструктор для перетворення із b в a, компілятор пробує наступний: `0 @ (b <=> a)` .
- ◇ Таким чином, зникає потреба писати комбінації, наприклад `operator@(ratio_t, int)` і `operator@(int, ratio_t)` – достатньо однієї.
- По замовчуванню створений оператор є constexpr і noexcept.

- Оператор <=> можна означити і вручну.
- Приклад – змінимо порядок порівняння членів:

```
struct C3{
    int a, b;
    //! Спочатку порівнюємо b, потім a
    auto operator<=>(const C3& rhs) const {
        if (auto cmp = b <=> rhs.b; cmp != 0){
            return cmp;
        } else{
            return a <=> rhs.a;
        }
    }
    bool operator==(const C3& rhs) const{
        return (a == rhs.a) && (b == rhs.b);
    }
};
```

- При цьому оператор == не створюється автоматично – за потреби його доводиться створювати вручну.
- Можна, звичайно, і так:

```
bool operator==(const C2& rhs) const = default;
```

- Інші явно визначені оператори порівняння будуть ігноруватися.

- У прикладі вище, для написання операції порівняння було використано <=> для членів.
- Однак, якщо оператори порівняння створюються автоматично із <=>, зворотного не відбувається, для багатьох інших типів код вище не скомпілюється.
- Звичайно, можна скористатися шаблонним метапрограмуванням:

```
template<typename T, typename U>
struct C4{     T a;     U b;          .....          };

template<typename T, typename U>
auto operator<=>(const C4<T,U>& lhs, const C4<T,U>& rhs) {
    if constexpr ( std::three_way_comparable_with<T,U>){
        return lhs <=> rhs;
    } else {
        assert(false && "Emulate <=> behaviour here");
#if 0
        // Citing https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0515r3.pd
        else if constexpr (/* can invoke a<b and a==b */)
            return a==b ? strong_ordering::equal : a<b ? strong_ordering::less :
                                                         strong_ordering::greater;
        else if constexpr (/* can invoke a==b */)
            return a == b ? strong_equality::equal : strong_equality::unequal;
        else if constexpr (is_same_v<T,U> && /* can invoke a.M <=> b.M
                        for each member M of T */) /* do that */;
#endif
```

- Однак, це довго, складно і легко помилитися.
- Кращий спосіб – скористатися бібліотечною об'єктом-функцією[10]:

```
template<typename T, typename U>
struct C5{
    T a;
    U b;

    bool operator==(const C5& rhs) const = default;
};

template<typename T, typename U>
auto operator<=>(const C5<T,U>& lhs, const C5<T,U>& rhs) {
    if (auto res = std::compare_three_way{}(lhs.a, rhs.a); res != 0)
        return res;
    return std::compare_three_way{}(lhs.b, rhs.b);
}
```

- Цей код не працюватиме, наприклад, якщо T=double, U=int. Причина – на наступних слайдах.

[10] https://en.cppreference.com/w/cpp/utility/compare/compare_three_way, застаріла версія – функція compare_3way(). Див. також https://en.cppreference.com/w/cpp/algorithm/lexicographical_compare_three_way.

- Код вище, для повного розуміння, потребує розібратися в математиці порівняння.
- Виділяють кілька категорій впорядкувань:
  - std::strong_ordering – строге впорядкування. Завжди є істинним лише рівно одне із порівнянь: a < b, a > b, a == b; гарантується, що якщо a == b, то f(a) == f(b) – еквівалентні елементи неможливо відрізнити.
    - Приклад – int чи std::string[11].
  - std::weak_ordering – слабке впорядкування. Завжди є істинним лише рівно одне із порівнянь: a < b, a > b, a == b; однак, якщо a == b, все ще може бути: f(a) != f(b) – еквівалентні елементи можна відрізнити.
    - Приклад – стрічка, що порівнюється нечутливо до регістра літер.
  - std::partial_ordering – часткове впорядкування. Жодне із наступних порівнянь може не бути істинним: a < b, a > b, a == b; якщо a == b, може бути: f(a) != f(b). Частина елементів є непорівняними.
    - Приклад – double, NaN != NaN.
- Кожна категорія описується відповідним класом із <compare>.

[11] Зауваження: порівняння стрічок не враховує capacity – використовуються лише так звані salient properties.

- **std::strong_ordering** може набувати значень (inline/static constexpr):
  - less – менше, "впорядкований перед".
  - equivalent та equal – рівність та еквівалентність для цієї категорії співпадають, не є впорядкованим ні перед ні позаду.
  - greater – більше, "впорядкований після".
- Порівнювати значення об'єкту std::strong_ordering із чимось, крім нуля – undefined behavior.
- Значення std::strong_ordering може неявно перетворюватися у дві інші категорії.
- **std::weak_ordering** може набувати значень less, equivalent та greater.
  - Варіант equal не передбачений.
- std::weak_ordering можна неявно перетворювати в std::partial_ordering.
- **std::partial_ordering**, окрім less, equivalent та greater може набувати:
  - unordered – непорівняний.
- Вбудований оператор <=> для floating point використовує саме це порівняння – NaN є непорівняними із будь-якими іншими, а додатній і від'ємний нуль – еквівалентні, але не рівні.

- Функції std::is_eq, std::is_neq, std::is_lt, std::is_gt, std::is_lteq, std::is_gteq – перетворюють результат тришляхового порівняння у порівняння, закодоване у назві, до прикладу:

  ```
  constexpr bool is_eq( std :: partial_ordering cmp ) noexcept;
  ```

-

- З врахуванням категорій, код вище:

```
template<typename T, typename U>
struct C6{
    T a;
    U b;

    bool operator==(const C6& rhs) const = default;
};

template<typename T, typename U>
auto operator<=>(const C6<T,U>& lhs, const C6<T,U>& rhs) ->
    typename std::common_type<
        decltype(std::compare_three_way{}(lhs.a, rhs.a)),
        decltype(std::compare_three_way{}(lhs.b, rhs.b))>::type
{
    if (auto res = std::compare_three_way{}(lhs.a, rhs.a); res != 0)
        return res;
    return std::compare_three_way{}(lhs.b, rhs.b);
}
```

Посилання

- https://ggulgulia.medium.com/c-20-three-way-comparison-operator-part-1-60e2946ae4a3, https://ggulgulia.medium.com/c-20-three-way-comparison-operator-part-2-fd520fb75e00, https://ggulgulia.medium.com/c-20-three-way-comparison-operator-part-3-143ac397b3f1, https://ggulgulia.medium.com/c-20-three-way-comparison-operator-part-4-6a8ea2c0a3f0, https://medium.com/codex/c-20-three-way-comparison-operator-part-5-e986b5ab29d9, https://medium.com/nerd-for-tech/c-20-three-way-comparison-operator-part-6-c66a173dcc99, https://ggulgulia.medium.com/c-20-three-way-comparison-operator-part-7-8de8186bd5fc, https://medium.com/nerd-for-tech/c-20-three-way-comparison-operator-ensure-backward-compatibility-

- https://devblogs.microsoft.com/cppblog/simplify-your-code-with-rocket-science-c20s-spaceship-operator/
- https://accu.org/journals/overload/26/147/revzin_2563/ – "Implementing the Spaceship Operator for Optional", синтаксично дещо застаріла, використовує функцію із чернеток стандарту: compare_3way, замість об'єкту-функції compare_three_way, категорії strong_equality і т.д.
- https://en.cppreference.com/w/cpp/language/default_comparisons, https://en.cppreference.com/w/cpp/language/operator_comparison#Three-way_comparison, https://en.cppreference.com/w/cpp/utility/compare/strong_ordering, https://en.cppreference.com/w/cpp/utility/compare/weak_ordering, https://en.cppreference.com/w/cpp/utility/compare/partial_ordering, https://en.cppreference.com/w/cpp/utility/compare/named_comparison_functions, https://en.cppreference.com/w/cpp/utility/compare/compare_three_way.
- https://www.foonathan.net/2018/07/ordering-relations-math/ – види впорядкувань.
- https://devblogs.microsoft.com/oldnewthing/20220516-52/?p=106661 – "How can I synthesize a C++20 three-way comparison from two-way comparisons?"
- https://quuxplusone.github.io/blog/2023/06/05/spaceship-ignore/ – "Make 'operator<=>' ignore a data member"

